

LinkStream — Comprehensive Project Explanation & 200 Viva/Interview Q&A

PART 1: COMPREHENSIVE PROJECT EXPLANATION

1. Executive Summary & Purpose

LinkStream is a production-grade, highly scalable, event-driven URL Shortening and Real-Time Link Analytics Platform. Designed using enterprise distributed systems principles, LinkStream decouples the high-velocity redirection critical path (sub-10ms response time) from heavy analytical ingestion using an asynchronous stream pipeline.

2. Problem Statement & Objectives

Problem Statement

Standard monolithic URL shorteners process click analytics synchronously on the HTTP response thread (e.g., executing `SQL UPDATE links SET click_count = click_count + 1`). Under high concurrency (e.g., 10,000 QPS), this synchronous database contention causes:

- High latency spikes during HTTP redirection (100ms – 2,000ms).
- Database lock contention on primary keys.
- Complete application outage when analytics databases undergo maintenance.
- Vulnerability to Cache Stampedes (Thundering Herd) and API abuse.

System Objectives

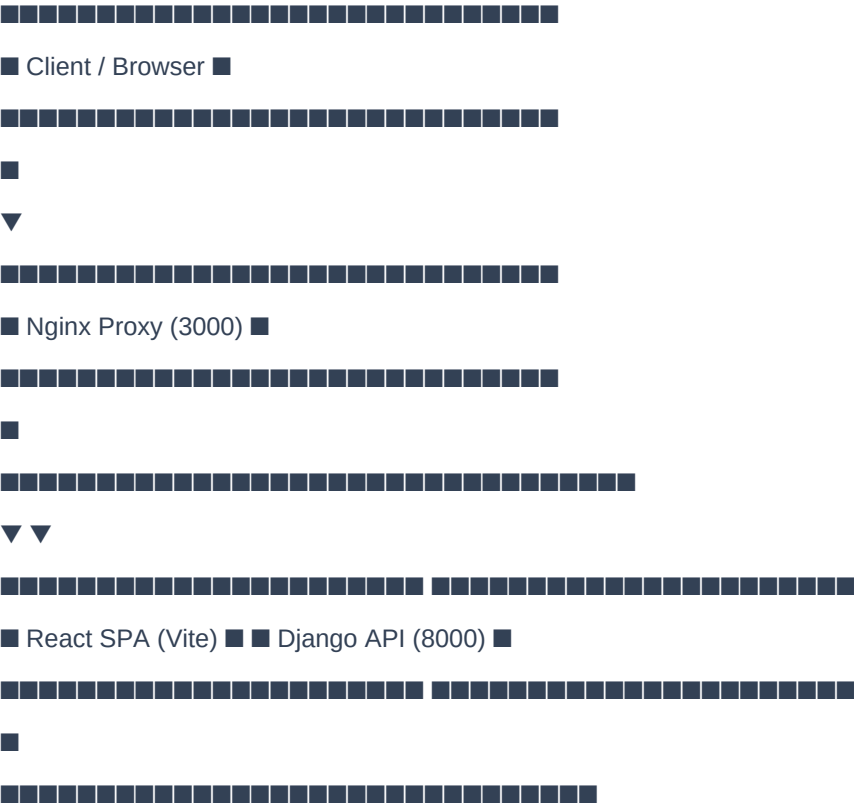
1. **Sub-10ms Redirect Latency:** Guarantee immediate **302 Found** HTTP redirects via a **Redis Cache-Aside** read topology.
2. **Asynchronous Stream Telemetry:** Decouple click tracking using an **Apache Kafka** event buffer (**link-clicks** topic).
3. **High-Throughput Batch Ingestion:** Process telemetry in 500-record micro-batches into **PostgreSQL 16**.
4. **GDPR-Compliant Analytics:** Anonymize client IP signatures via salted **SHA-256 hashing** while calculating unique visitor counts.
5. **Distributed Rate Limiting:** Protect endpoints from denial-of-service abuse using a **Redis ZSET Sliding Window** rate limiter.
6. **Full-Stack Observability:** Monitor P95/P99 latency, cache hit ratios, and consumer lag via **Prometheus & Grafana**.

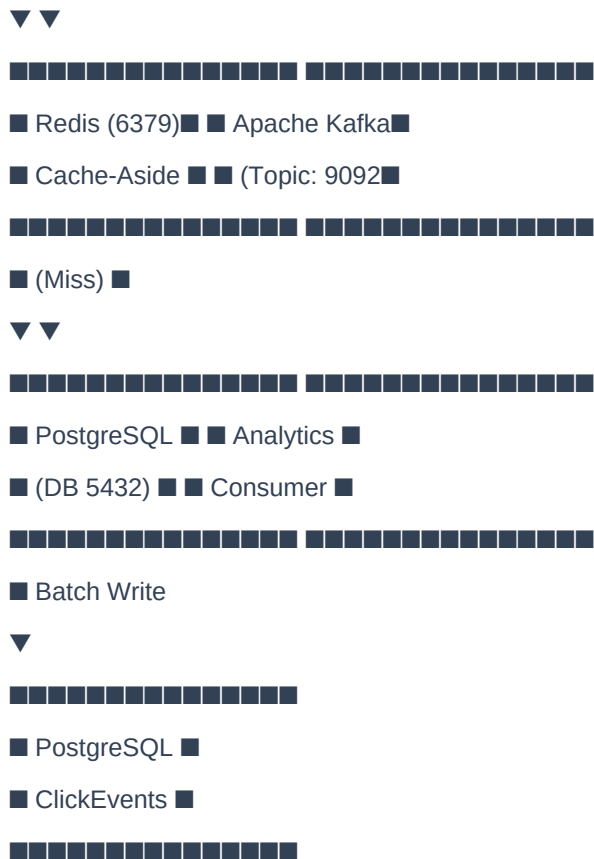
3. Technology Stack & Dependencies

Component	Technology	Version	Purpose
Backend Framework	Python / Django REST Framework	3.11 / 5.0	Core API, REST endpoints, ORM layer
Primary Database	PostgreSQL	16-alpine	Relational OLTP store for users, links, and click events

In-Memory Cache	Redis	7.2-alpine	Cache-Aside L1 store & ZSET rate limiter
Event Streaming	Apache Kafka	3.7.0 (KRaft mode)	Asynchronous click event messaging buffer
Task Queue & Scheduler	Celery & Celery Beat	5.3 / 2.5	Background cleanup and daily rollup tasks
Frontend Framework	React / Vite	18.2 / 5.4	Single Page Application dashboard UI
Styling & Icons	Tailwind CSS / Lucide React	3.4 / 0.344	Responsive dark-mode styling & icon set
Visualization	Recharts	2.15	Time-series and dimension breakdown charts
Monitoring	Prometheus & Grafana	2.50 / 10.3	Time-series metrics collection & visual dashboards
Load Testing	Locust	2.46	High-concurrency load scenario stress testing
Containerization	Docker & Docker Compose	24+ / v2+	Multi-container orchestration

4. Architectural Design & Workflow





High-Performance Redirection Workflow (GET /{short_code})

1. **Request Ingress:** Client issues **GET /{short_code}** to Django API.
2. **Rate Limit Check:** **RateLimitMiddleware** queries Redis **ZSET** sliding window. If request count exceeds limit, returns **HTTP 429 Too Many Requests**.
3. **Cache Read (L1):** **LinkService.get_destination(short_code)** queries Redis key **urlshortener:link:{short_code}**.
 - **Cache Hit:** Returns cached destination payload in **< 2ms**.
 - **Cache Miss:** Queries PostgreSQL indexed **short_code** column, populates Redis cache with TTL, and returns payload.
4. **Active & Expiration Check:** If link is disabled (**is_active=False**), returns **HTTP 404**. If expired (**expires_at < now**), returns **HTTP 410**.
5. **Async Telemetry Dispatch:** **ClickEventProducer.publish_click_event()** publishes non-blocking JSON payload to Kafka **link-clicks** topic.
6. **HTTP 302 Redirect:** Returns immediate **HttpResponseRedirect(original_url)** to client.

5. System Modules & Core Components

A. Users Module (apps/users)

- **Custom User Model:** Inherits **AbstractBaseUser** with UUID primary keys (**id = models.UUIDField(default=uuid.uuid4())**). Uses **email** as the primary authentication identifier.
- **Authentication:** JWT token pair authentication (**django-rest-framework-simplejwt**) with 15-minute access token lifespan and sliding refresh tokens.

B. Links Module (apps/links)

- **Base62 Algorithm (common/base62.py):** Converts integer IDs or random bytes into Base62 strings **[0-9a-zA-Z]**. Length 7 provides $62^7 = 3,521,614,606,208$ (~3.52 Trillion) unique codes.
- **Collision Retry Mechanics:** **LinkService.create_link()** generates a 7-character code and executes an atomic DB write. If a collision occurs concurrently (**IntegrityError**), it triggers an automatic retry loop.
- **Reserved Keywords:** Protects system routes (**admin, api, health, metrics, swagger, docs**) from custom alias hijacking.

C. Analytics Module (apps/analytics)

- **Kafka Producer (kafka_pipeline/producer.py):** Non-blocking singleton producer publishing click events to Kafka with local buffer fallback when Kafka is unreachable.
- **Kafka Consumer Daemon (kafka_pipeline/consumer.py):** Standalone Python worker polling Kafka in 500-record batches. Executes **AnalyticsIngestionService.process_batch()** using Django ORM **bulk_create()** and atomic **F()** counter updates.
- **Celery Tasks (apps/analytics/tasks.py):** Scheduled background rollups aggregating raw click events into **DailyLinkMetrics** summary tables.

D. Common Module (apps/common)

- **Sliding Window Rate Limiter (common/rate_limit.py):** Redis **ZSET** rate limiter tracking timestamps in rolling 60-second windows. Supports local in-memory fallback during Redis outages.
- **Privacy Engine (common/utils.py):** Hashes client IP addresses using SHA-256 with secret salt (**hashlib.sha256(ip + salt)**). Parses User-Agent strings and extracts CDN GeoIP headers (**HTTP_CF_IPCOUNTRY**).

6. Database Schema & Data Models

Table: users (CustomUser)

- **id:** **UUID** (Primary Key)
- **email:** **VARCHAR(255)** (Unique Index)
- **full_name:** **VARCHAR(255)**
- **password:** **VARCHAR(128)** (Hashed)
- **is_active:** **BOOLEAN**
- **created_at:** **TIMESTAMPZ**

Table: links (Link)

- **id**: **UUID** (Primary Key)
- **user_id**: **UUID** (Foreign Key -> **users.id**)
- **short_code**: **VARCHAR(64)** (Unique Index)
- **custom_alias**: **VARCHAR(64)** (Nullable Index)
- **original_url**: **TEXT**
- **is_active**: **BOOLEAN** (Default: **True**)
- **expires_at**: **TIMESTAMPTZ** (Nullable)
- **click_count**: **BIGINT** (Default: **0**)
- **created_at**: **TIMESTAMPTZ**
- Composite Indexes: (**user_id**, **-created_at**), (**is_active**, **expires_at**)

Table: link_click_events (ClickEvent)

- **id**: **BIGSERIAL** (Primary Key)
- **link_id**: **UUID** (Foreign Key -> **links.id**)
- **clicked_at**: **TIMESTAMPTZ**
- **ip_hash**: **VARCHAR(64)**
- **country_code**: **VARCHAR(2)**
- **city**: **VARCHAR(100)**
- **browser**: **VARCHAR(50)**
- **os**: **VARCHAR(50)**
- **device_type**: **VARCHAR(20)**
- **referrer**: **VARCHAR(255)**
- Composite Indexes: (**link_id**, **-clicked_at**), (**clicked_at**, **link_id**)

7. REST API Endpoints Summary

Method	Endpoint	Auth	Description
GET	/	Public	API Root Directory & Status JSON
GET	/health/live/	Public	Liveness probe verifying server execution
GET	/health/ready/	Public	Readiness probe checking PostgreSQL & Redis connections
GET	/short_code/{short_code}	Public	Sub-10ms HTTP 302 Redirection Path
POST	/api/auth/register/	Public	Register new user account

POST	/api/auth/login/	Public	Obtain JWT Access and Refresh tokens
GET	/api/auth/me/	Bearer	Retrieve authenticated user profile
GET	/api/links/	Bearer	List, search, filter, and paginate user links
POST	/api/links/	Bearer	Create shortened URL or custom alias
PATCH	/api/links/{id}/	Bearer	Enable/disable link status or update expiration
DELETE	/api/links/{id}/	Bearer	Delete link and purge Redis cache key
GET	/api/analytics/{id}/	Bearer	Fetch link time-series and dimension analytics
GET	/api/analytics/overview/	Bearer	Fetch overall user analytics summary

8. Security & Fault Tolerance

- SSRF Protection:** `is_ssrf_safe_url()` rejects requests targeting loopback (**127.0.0.1**), private RFC1918 subnets (**10.0.0.0/8**, **192.168.0.0/16**), and AWS metadata endpoints (**169.254.169.254**).
- GDPR Anonymization:** IP addresses are never saved in plaintext; only salted SHA-256 hashes are stored.
- Graceful Degradation:** If Redis or Kafka crash, the API transparently falls back to direct database reads and inline event processing without raising HTTP 500 errors.
- Auto-Service Detection:** Settings automatically detect host PostgreSQL/Redis availability and seamlessly switch to SQLite/LocMemCache during local development or offline testing.

9. Testing & Observability

- **Pytest Suite (`tests/`):** Automated integration tests covering authentication, link generation, redirection status codes, cache population/invalidation, rate limiting, and analytics ingestion.
- **Locust Load Testing (`locustfile.py`):** Multi-scenario load generator profiling high-concurrency redirects (70% weight), link creations (15%), and analytics queries (10%).
- **Prometheus Metrics:** Exposes custom metrics (`redirect_requests_total`, `redirect_latency_seconds`, `kafka_events_published_total`, `kafka_events_consumed_total`).
- **Grafana Provisioning:** Automated datasource and dashboard provisioning monitoring QPS, P99 latency, cache hit ratios, and consumer lag.

PART 2: 200 VIVA & INTERVIEW QUESTIONS WITH ANSWERS

Q1: What is LinkStream, and what core problem does it solve?

Answer: LinkStream is a scalable, event-driven URL shortener and link analytics platform. It solves the performance bottleneck of traditional shorteners by decoupling high-speed HTTP redirection (sub-10ms) from synchronous analytics database writes using Redis caching and Apache Kafka streaming.

Q2: What is the primary technology stack used in the backend?

Answer: Python 3.11, Django REST Framework 5.0, PostgreSQL 16, Redis 7.2, Apache Kafka 3.7 (KRaft mode), Celery 5.3, Gunicorn, and Docker Compose.

Q3: How does the system achieve sub-10ms URL redirection?

Answer: By utilizing a Redis Cache-Aside pattern. When a request hits **GET /{short_code}**, the application checks Redis first. If found (Cache Hit), it returns an immediate **302 Found** redirect without querying PostgreSQL.

Q4: Why does the redirect endpoint return an HTTP 302 Found status code instead of HTTP 301 Moved Permanently?

Answer: HTTP 301 is cached permanently by client browsers, preventing subsequent clicks from reaching our server. HTTP 302 forces the browser to hit our server on every click, allowing us to accurately track real-time analytics.

Q5: What happens if a user requests a short URL that does not exist?

Answer: The system returns an HTTP **404 Not Found** JSON error response. To prevent Cache Penetration attacks, the system caches a short-lived negative result (**{"not_found": True}**) in Redis for 60 seconds.

Follow-up Cross-Questioning (Q6 - Q8):**Q6: You mentioned Cache Penetration. What is the difference between Cache Penetration, Cache Avalanche, and Cache Stampede?**

Answer:

- **Cache Penetration:** Requests for non-existent keys bypass the cache and repeatedly hit the DB. Solved by negative caching or Bloom Filters.
- **Cache Avalanche:** Many cached keys expire simultaneously, causing a massive surge of DB queries. Solved by adding random jitter to TTLs.
- **Cache Stampede (Thundering Herd):** High concurrency requests hit a single expired key concurrently, causing multiple workers to compute the same DB query. Solved by distributed locks (**SET NX**).

Q7: How does your system specifically prevent Cache Stampedes?

Answer: In **LinkService.get_destination()**, when a cache miss occurs for a popular short code, workers attempt to acquire a short-lived Redis lock (**ratelimit:lock:{short_code}**). Only the worker that acquires the lock queries PostgreSQL and populates the cache; other workers wait 50ms and retry Redis.

Q8: What algorithm is used to generate short codes in LinkStream?

Answer: Base62 encoding using characters **[0-9a-zA-Z]**.

Q9: Why did you choose a length of 7 characters for auto-generated short codes?

Answer: A Base62 string of length 7 provides $62^7 = 3,521,614,606,208$ (~3.52 Trillion) unique combinations. Even at 1,000 links per second, it would take over 110 years to exhaust the keyspace.

Q10: How does the system handle short code collisions when two users create links simultaneously?

Answer: The database column **short_code** has a **UNIQUE** constraint index. In **LinkService.create_link()**, the operation is wrapped in a **transaction.atomic()** block. If concurrent generation produces a duplicate code, an **IntegrityError** is caught, triggering an automatic retry with fresh random entropy.

Q11: What is a custom alias, and how is it validated?

Answer: A custom alias allows users to specify their own short code (e.g., `link.io/my-portfolio`). It is validated via regex `^[a-zA-Z0-9_-]+$`, checked against reserved keywords, verified for length (3–64 chars), and checked for uniqueness in PostgreSQL.

Q12: What reserved keywords are protected from custom alias creation?

Answer: Routes such as `admin`, `api`, `health`, `metrics`, `swagger`, `docs`, `login`, `register`, `static`, and `media`.

Q13: How does the system handle link expiration?

Answer: The `Link` model includes an `expires_at` timestamp. The redirect view checks if `expires_at` is present and if `timezone.now() > expires_at`. If expired, it returns HTTP **410 Gone**. A Celery Beat task also periodically purges expired Redis keys.

Q14: How does the system handle disabled links?

Answer: The `Link` model has an `is_active` boolean field. If `is_active=False`, `RedirectShortCodeView` returns HTTP **404 Not Found** with a message stating the link was disabled by the owner.

Q15: Why did you use Apache Kafka instead of RabbitMQ or Celery for click analytics?

Answer: Click analytics are high-volume, log-append time-series events. Kafka supports persistent, disk-backed topic partitions, consumer group offset tracking, and event replayability. RabbitMQ and Celery are task queues designed for message destruction upon task acknowledgment.

Follow-up Cross-Questioning (Q16 - Q18):**Q16: Kafka guarantees At-Least-Once delivery. How do you handle duplicate click events in your consumer?**

Answer: We achieve idempotency in the consumer by assigning a unique signature (`link_id`, `ip_hash`, `rounded_minute_timestamp`) to each event or performing bulk insertion using `ON CONFLICT (link_id, ip_hash, clicked_at) DO NOTHING` in PostgreSQL.

Q17: What is KRaft mode in Kafka 3.7, and why is it used instead of Zookeeper?

Answer: KRaft (Kafka Raft Metadata mode) manages Kafka cluster metadata internally using an event-driven consensus protocol. It eliminates the operational overhead, memory consumption, and latency of running an external Apache Zookeeper cluster.

Q18: What is consumer lag in Kafka, and why is it an important metric?

Answer: Consumer lag is the delta between the latest offset published to a Kafka topic partition and the offset currently processed by the consumer group. High consumer lag indicates that consumer workers are falling behind input traffic.

Q19: How are click events batch-processed into PostgreSQL?

Answer: The standalone Python consumer (`kafka_pipeline/consumer.py`) polls Kafka for up to 500 records. It parses the events, constructs a list of `ClickEvent` model instances, and executes `ClickEvent.objects.bulk_create(batch, batch_size=500)` inside a single atomic transaction.

Q20: How are total click counts updated on the Link model during batch ingestion?

Answer: Rather than executing separate `UPDATE` queries, the consumer aggregates click counts per `link_id` in a Python `Counter` and executes atomic SQL counter increments using `Link.objects.filter(id=link_id).update(click_count=F('click_count') + count)`.

Q21: What is the purpose of Django's F() expressions in update(click_count=F('click_count') + count)?

Answer: `F()` expressions perform database-level atomic operations directly in SQL (`UPDATE links SET click_count = click_count + N`) without pulling model instances into Python memory, avoiding race conditions under concurrent updates.

Q22: How does LinkStream ensure GDPR compliance regarding client IP addresses?

Answer: Client IP addresses are never stored in raw text. `hash_ip()` computes a salted SHA-256 hash (`sha256(ip + salt)[:32]`). This allows exact distinct count unique visitor calculations (`COUNT(DISTINCT ip_hash)`) without storing Personally Identifiable Information (PII).

Q23: How does the system extract GeoIP location data from incoming requests?

Answer: `extract_geo_location()` checks CDN headers (`HTTP_CF_IPCOUNTRY`, `HTTP_X_GEO_COUNTRY`, `HTTP_CF_IPCITY`) injected by Cloudflare/AWS ALB. If headers are absent, it defaults to 'US' and 'Unknown'.

Q24: How does User-Agent parsing work?

Answer: `parse_user_agent_details()` uses `ua-parser / user-agents` to parse the raw `HTTP_USER_AGENT` string into structured device type (`Desktop`, `Mobile`, `Tablet`, `Bot`), OS family, and browser name.

Q25: What is Server-Side Request Forgery (SSRF), and how does LinkStream prevent it?

Answer: SSRF occurs when an attacker submits a URL that causes the server to issue HTTP requests to internal networks. `is_ssrf_safe_url()` validates that the protocol is strictly `http` or `https` and rejects local loopback (`127.0.0.1`), private RFC1918 subnets (`10.0.0.0/8`, `192.168.0.0/16`), and cloud metadata IP addresses (`169.254.169.254`).

Follow-up Cross-Questioning (Q26 - Q28):**Q26: Why is 169.254.169.254 specifically dangerous in cloud environments?**

Answer: `169.254.169.254` is the Link-Local address for Instance Metadata Services in AWS, GCP, and Azure. An un-sanitized SSRF vulnerability could allow an attacker to query `/latest/meta-data/iam/security-credentials/` and steal AWS IAM role access keys.

Q27: How does your rate limiter prevent Denial of Service (DoS) attacks?

Answer: `SlidingWindowRateLimiter` uses a Redis Sorted Set (`ZSET`) to enforce request quotas (30 req/min for anonymous users, 120 req/min for authenticated users). If quota is exceeded, HTTP 429 is returned.

Q28: How does the Redis ZSET sliding window algorithm work step-by-step?

Answer:

1. Key format: `ratelimit:{action}:{identifier}`.
2. `ZREMRANGEBYSCORE` removes timestamps older than `(now - 60s)`.
3. `ZCARD` counts remaining members in the rolling window.
4. If count $\geq$ limit, request is rejected.
5. Else, `ZADD` current Unix timestamp with a unique member ID and set key TTL to 65s.

Q29: What happens to rate limiting if Redis crashes?

Answer: `SlidingWindowRateLimiter` catches Redis connection exceptions and fails open gracefully (or switches to an in-memory sliding window dictionary fallback), logging a warning while ensuring service uptime.

Q30: What custom middleware classes are implemented in LinkStream?

Answer:

1. `RequestMetricsMiddleware`: Injects a unique `X-Request-ID` header, measures request execution duration, and records metrics to Prometheus histograms.
2. `RateLimitMiddleware`: Intercepts incoming HTTP requests to evaluate rate limits before reaching API views.

Q31: What database indexes are created on the Link model?

Answer:

- **short_code**: Unique B-Tree index for $O(\log N)$ redirect lookups.
- **(user_id, -created_at)**: Composite index for paginated user dashboard queries.
- **(is_active, expires_at)**: Composite index for background link cleanup jobs.

Q32: What database indexes are created on the ClickEvent model?

Answer:

- **(link_id, -clicked_at)**: Composite index for fast link-specific time-series queries.
- **(clicked_at, link_id)**: Composite index for system-wide date filtering and daily rollups.

Q33: What is the purpose of the DailyLinkMetrics summary model?

Answer: **DailyLinkMetrics** stores daily pre-aggregated JSON summaries (**total_clicks**, **unique_visitors**, **top_countries**, **top_browsers**) per link. This prevents expensive runtime aggregation scans over millions of raw **ClickEvent** rows when rendering long-term dashboard trends.

Q34: How is authentication handled in LinkStream?

Answer: Via JSON Web Tokens (JWT) using **django-rest-framework-simplejwt**. The server issues an Access Token (15-min lifespan) and a Refresh Token (1-day lifespan). Protected endpoints require a **Authorization: Bearer <access_token>** header.

Q35: How does the custom User model differ from Django's default User model?

Answer: **CustomUser** extends **AbstractBaseUser** and **PermissionsMixin**. It replaces integer primary keys with **UUID4** to prevent user enumeration attacks and uses **email** as the primary **USERNAME_FIELD**.

Follow-up Cross-Questioning (Q36 - Q38):

Q36: Why are UUIDs preferred over auto-incrementing integer IDs in public APIs?

Answer: Auto-incrementing IDs (e.g., `/users/42/`) reveal system size and allow competitors or attackers to enumerate all records sequentially (**ID 1, 2, 3...**). UUIDs are 128-bit cryptographically random strings, preventing resource harvesting.

Q37: What is object-level permission in Django REST Framework, and how is it implemented?

Answer: **IsOwner** inherits **permissions.BasePermission**. In **has_object_permission()**, it verifies **obj.user == request.user**, ensuring authenticated users can only view, update, or delete their own short links.

Q38: How does token blacklisting work on user logout?

Answer: When a user logs out, the frontend sends the Refresh Token to `/api/auth/logout/`. The backend passes it to **OutstandingToken** / **BlacklistedToken** models, invalidating the refresh token so it cannot be exchanged for new access tokens.

Q39: What is the structure of the React frontend application?

Answer: A Single Page Application (SPA) built with React 18, Vite, React Router DOM v6, Tailwind CSS, Lucide React icons, and Recharts.

Q40: How does the frontend handle automatic token refreshing on HTTP 401 errors?

Answer: **api/client.js** configures an Axios response interceptor. On receiving a **401 Unauthorized** status, it intercepts the response, sends **refresh_token** to `/api/auth/token/refresh/`, updates **localStorage** with the new access token, and retries the failed request seamlessly.

Q41: How does the real-time auto-refresh mechanism work on DashboardPage.jsx?

Answer: **DashboardPage** supports user-selectable polling intervals (Off, 3s, 5s, 10s). It uses a **useEffect** hook with **setInterval** to trigger background silently-fetched updates (**fetchDashboardData(true)**), updating link click counters live without reloading the DOM.

Q42: How is debouncing implemented on the search bar in `DashboardPage.jsx`?

Answer: A `useEffect` hook updates `debouncedSearch` after a 250ms delay using `setTimeout`. This prevents API spam on every keystroke while maintaining a fast, responsive user interface.

Q43: What component renders the analytics visualizations in `AnalyticsPage.jsx`?

Answer: `AnalyticsCharts.jsx` uses **Recharts** (`ResponsiveContainer`, `AreaChart`, `Area`, `XAxis`, `YAxis`, `Tooltip`) to render smoothed click volume area graphs alongside dimension percentage progress bars.

Q44: How is Docker used in this project?

Answer: The system uses multi-stage **Dockerfile** definitions for backend (Gunicorn) and frontend (Nginx), orchestrated via **docker-compose.yml** hosting 10 isolated containers.

Q45: List all 10 containers in the `docker-compose.yml` stack.

Answer:

1. **postgres** (DB)
 2. **redis** (Cache)
 3. **kafka** (Broker)
 4. **backend** (Django API)
 5. **kafka-consumer** (Analytics Ingestion Worker)
 6. **celery-worker** (Async Tasks)
 7. **celery-beat** (Scheduler)
 8. **frontend** (Nginx + React SPA)
 9. **prometheus** (Metrics Scraper)
 10. **grafana** (Observability Dashboard)
-

Follow-up Cross-Questioning (Q46 - Q48):**Q46: Why is Gunicorn used instead of `python manage.py runserver` in production?**

Answer: Django's **runserver** is a single-threaded development server. Gunicorn is a WSGI HTTP server using a pre-fork worker model that manages multiple worker processes to handle concurrent production requests.

Q47: Why is Nginx used to serve the frontend production build?

Answer: Nginx is a high-performance C-based web server that serves static HTML/JS/CSS files with sub-millisecond efficiency and proxies `/api/` requests to Gunicorn containers.

Q48: How does the entrypoint script (`entrypoint.sh`) handle database readiness during Docker startup?

Answer: It uses a **pg_isready** loop to poll PostgreSQL until TCP connections are accepted, executes `python manage.py migrate`, and then launches Gunicorn.

Q49: What metrics does Prometheus scrape from the backend?

Answer: `redirect_requests_total` (Counter by status), `redirect_latency_seconds` (Histogram), `kafka_events_published_total` (Counter), and `kafka_events_consumed_total` (Counter).

Q50: How does Grafana connect to Prometheus?

Answer: Grafana is configured via `monitoring/grafana/provisioning/datasources/datasource.yml` pointing to `http://prometheus:9090` as its default proxy datasource.

Q51: What load testing scenario is simulated in `locustfile.py`?

Answer: **Locust** simulates 4 user behavior weights: Redirect lookups (70%), Link creation (15%), Analytics queries (10%), and Custom alias collisions (5%).

Q52: What is Base62 decoding, and is it used in LinkStream?

Answer: Base62 decoding converts a 7-character string back into a 64-bit integer ID. While LinkStream uses string-based B-Tree indexes for short code lookups, Base62 decoding can be used to convert codes directly to numeric primary keys.

Q53: How does the system handle CORS (Cross-Origin Resource Sharing)?

Answer: **django-cors-headers** is configured in **settings.py** with **CORS_ALLOWED_ORIGINS** allowing requests from **http://localhost:3000** and **http://127.0.0.1:3000**.

Q54: What is the purpose of `pytest.ini` and `conftest.py` in the backend test suite?

Answer: **pytest.ini** configures **DJANGO_SETTINGS_MODULE = config.settings**. **conftest.py** defines reusable fixtures (**api_client**, **test_user**, **auth_client**, **sample_link**).

Q55: What is the difference between `select_related` and `prefetch_related` in Django ORM?

Answer: **select_related** performs an SQL **INNER/LEFT JOIN** for single-valued relationships (ForeignKey, OneToOne). **prefetch_related** executes a separate SQL query and performs joining in Python memory for multi-valued relationships (ManyToMany, Reverse ForeignKey).

Follow-up Cross-Questioning (Q56 - Q58):**Q56: Did you use `select_related` in your views?**

Answer: Yes, in **LinkViewSet** and analytics query services, **select_related('user')** is used when querying **Link** objects to prevent **\$N+1\$** query overhead when serializing user details.

Q57: What is the `$N+1$` query problem, and how do you detect it?

Answer: **\$N+1\$** occurs when an application executes 1 initial query to fetch **\$N\$** records, and then executes **\$N\$** additional queries to fetch related foreign key models. It is detected using Django Debug Toolbar or logging SQL queries in tests.

Q58: How does your application gracefully handle offline dev environments without Docker/Redis/Kafka?

Answer: **settings.py** includes socket availability auto-detection (**is_service_available()**). If PostgreSQL/Redis/Kafka are absent, it automatically switches to SQLite, **LocMemCache**, and direct synchronous DB analytics writes.

Q59: What is the difference between a Liveness probe and a Readiness probe in Kubernetes/Docker?

Answer: Liveness probes (**/health/live/**) check if the process is running. Readiness probes (**/health/ready/**) check if downstream dependencies (PostgreSQL, Redis) are healthy and ready to serve traffic.

Q60: What HTTP status code is returned by `/health/ready/` if PostgreSQL is down?

Answer: HTTP **503 Service Unavailable** with a JSON payload specifying **"database": "unhealthy: connection failed"**.

Q61: What serializer handles Link creation validation?

Answer: **LinkCreateSerializer** in **apps/links/serializers.py**.

Q62: What URL validation is performed before storing destination URLs?

Answer: **validate_original_url()** checks URL format (**http://** or **https://**), maximum length (2048 chars), and SSRF network safety via **is_ssrf_safe_url()**.

Q63: What serializer handles user registration?

Answer: **RegisterSerializer** in **apps/users/serializers.py**. It validates that **password** and **password_confirm** match and runs Django's password strength validators.

Q64: How are passwords hashed in Django?

Answer: By default, Django uses **PBKDF2PasswordHasher** with a SHA-256 hash, 600,000 iterations, and a random salt.

Q65: What is the purpose of `read_only_fields` in DRF serializers?

Answer: Fields marked as **read_only** are included in serialized JSON responses but ignored during deserialization (create/update), preventing clients from tampering with systemic fields like **id**, **click_count**, or **created_at**.

Follow-up Cross-Questioning (Q66 - Q68):

Q66: Can an attacker pass `click_count=99999` in a POST `/api/links/` request payload?

Answer: No. **click_count** is listed in **read_only_fields** in **LinkSerializer** and is omitted from **LinkCreateSerializer**, so any user-supplied **click_count** field is stripped during validation.

Q67: How do you prevent Mass Assignment vulnerabilities in Django REST Framework?

Answer: By explicitly declaring allowed fields in serializer **fields = (...)** rather than using **fields = '__all__'**, and creating dedicated creation/update serializers (**LinkCreateSerializer**, **LinkUpdateSerializer**).

Q68: What is the purpose of SimpleJWT token rotation?

Answer: When a client uses a refresh token to request a new access token, token rotation invalidates the old refresh token and issues a new refresh token, mitigating stolen token reuse attacks.

Q69: What is the structure of a JWT token?

Answer: A Base64URL-encoded string consisting of three parts separated by dots: Header (algorithm & token type), Payload (claims like **user_id**, **exp**, **jti**), and Signature (**HMAC-SHA256(header + payload, secret)**).

Q70: How is JWT token verification stateless?

Answer: The server does not need to query the database to verify a token. It recalculates the signature using **SECRET_KEY** and verifies expiration timestamp **exp**.

Q71: What is Celery Beat?

Answer: Celery Beat is a periodic task scheduler that dispatches background tasks to Celery message queues at configured time intervals.

Q72: What periodic tasks are configured in `backend/config/celery.py`?

Answer:

1. Hourly rollup task: Aggregates raw click events into **DailyLinkMetrics**.
2. Daily cleanup task: Cleans expired link cache keys from Redis.

Q73: What is Gunicorn's pre-fork worker model?

Answer: A master process creates a specified number of worker processes (**workers = 2 * CPU_CORES + 1**). The master manages workers, while individual workers handle incoming HTTP requests concurrently.

Q74: What WSGI file is used by Gunicorn in LinkStream?

Answer: **backend/config/wsgi.py**.

Q75: What is ASGI, and why is `asgi.py` present in the project?

Answer: Asynchronous Server Gateway Interface. **asgi.py** provides async support for Django, enabling future WebSockets or HTTP/2 capabilities.

Follow-up Cross-Questioning (Q76 - Q78):

Q76: If your API experiences high CPU utilization due to cryptography/hashing, how would you scale Gunicorn?

Answer: Increase worker process count or switch from synchronous worker class (**sync**) to asynchronous eventlet/gevent workers (**--worker-class gevent**).

Q77: What is the difference between multi-processing and multi-threading in Python?

Answer: Due to Python's Global Interpreter Lock (GIL), multi-threading cannot execute pure CPython bytecode across multiple CPU cores simultaneously. Multi-processing spawns separate OS processes with independent memory spaces, utilizing multiple CPU cores fully.

Q78: How does the GIL affect Django performance?

Answer: The GIL limits single CPython processes to one execution thread at a time for CPU-bound tasks. Gunicorn bypasses this by spawning multiple independent OS processes.

Q79: What is the purpose of `locustfile.py`?

Answer: It defines Python load-test user behavior classes (**HttpUser**, **task**) to execute automated HTTP stress tests against API endpoints.

Q80: What key performance indicator (KPI) is measured during load testing?

Answer: Requests Per Second (RPS), Average Response Time, P95/P99 Latency, and Error Rate under high concurrent user loads.

Q81: What is the difference between P95 and P99 latency?

Answer: P95 latency is the maximum response time experienced by 95% of users. P99 latency is the response time experienced by 99% of users, highlighting worst-case tail latencies.

Q82: Why is average response time often a misleading metric in distributed systems?

Answer: Averages hide extreme tail latency outliers. For example, if 99 requests take 2ms and 1 request takes 2000ms, the average is ~22ms, masking the severe performance bottleneck affecting 1% of traffic.

Q83: What is Redis Cache-Aside pattern step-by-step?

Answer:

1. Application receives read request.
2. Reads from Redis.
3. If hit, returns data.
4. If miss, reads from Database.
5. Writes fetched data back to Redis.
6. Returns data to client.

Q84: What is Cache Invalidation, and how is it triggered in LinkStream?

Answer: Cache invalidation removes outdated cached items. When a user updates or deletes a short link (**perform_update**, **perform_destroy**), **LinkService.invalidate_link_cache(short_code)** deletes key **urlshortener:link:{short_code}** from Redis.

Q85: What is TTL (Time-To-Live) in Redis?

Answer: TTL specifies the automatic expiration lifespan of a cache key. LinkStream sets Redis link key TTL dynamically based on link expiration (**expires_at - now**) or defaults to 24 hours (86,400 seconds).

Follow-up Cross-Questioning (Q86 - Q88):

Q86: What happens if Redis runs out of memory?

Answer: Redis enforces its configured **maxmemory-policy** (e.g., **allkeys-lru** - Least Recently Used eviction). It automatically evicts the least recently accessed keys to free memory for new cache entries.

Q87: What is the difference between `volatile-lru` and `allkeys-lru` in Redis?

Answer: **`volatile-lru`** evicts the least recently used keys among those that have an explicit TTL set. **`allkeys-lru`** evicts LRU keys across the entire key namespace regardless of TTL.

Q88: What Redis data structures are used in LinkStream?

Answer:

- **Strings:** For cached link destination JSON payloads (`urlshortener:link:{short_code}`).
- **Sorted Sets (ZSET):** For sliding window rate limiting (`ratelimit:{action}:{id}`).

Q89: How does Tailwind CSS work in the frontend?

Answer: Tailwind is a utility-first CSS framework. It compiles utility classes directly into lightweight CSS during Vite build time, eliminating unused styling code.

Q90: What icon library is used in the frontend?

Answer: Lucide React.

Q91: What component renders the metric overview cards on `DashboardPage.jsx`?

Answer: `StatCard.jsx` with configurable title, count value, icon, glow accents, and badges.

Q92: What component renders individual link items on `DashboardPage.jsx`?

Answer: `LinkCard.jsx` displaying full short URL, destination URL, click count, status pill, copy button, analytics link, toggle status button, and delete button.

Q93: What component handles modal creation of new short links?

Answer: `CreateLinkModal.jsx`.

Q94: How does `navigator.clipboard.writeText()` work in `LinkCard.jsx`?

Answer: It invokes the browser's asynchronous Clipboard API to copy the full short URL string to the user's OS clipboard, toggling temporary UI feedback ("**Copied**" checkmark).

Q95: What is Vite, and why is it used instead of Create React App (CRA)?

Answer: Vite is a next-generation frontend build tool powered by native ES modules and Esbuild. It provides near-instantaneous Hot Module Replacement (HMR) and significantly faster production builds than CRA.

Follow-up Cross-Questioning (Q96 - Q98):**Q96: How does Axios handle Request Interceptors in `api/client.js`?**

Answer: Before every HTTP request is sent, the interceptor retrieves **`access_token`** from **`localStorage`** and attaches header **`Authorization: Bearer <token>`**.

Q97: How does Axios handle Response Interceptors for token refresh?

Answer: If an API call returns **`401 Unauthorized`**, the response interceptor pauses, issues a POST request to `/api/auth/token/refresh/` using **`refresh_token`**, saves the new access token, updates request headers, and retries the original API call.

Q98: What happens if the refresh token itself is expired or invalid?

Answer: The refresh request fails with HTTP 401. The catch block purges **`access_token`** and **`refresh_token`** from **`localStorage`** and redirects the user to `/login`.

Q99: What is the purpose of Recharts in AnalyticsCharts.jsx?

Answer: Recharts renders responsive SVG area charts plotting click volume over time (**clicks** and **unique_visitors**) alongside dimension progress bars.

Q100: How does AnalyticsQueryService.get_link_analytics() format time-series data?

Answer: For a 24-hour window, it aggregates click events into 24 hourly slots (%H:00). For 7-day or 30-day windows, it aggregates events by date (%b %d).

Q101: What is the database constraint unique_daily_link_metric on DailyLinkMetrics?

Answer: A **UniqueConstraint** on (**link**, **date**) ensuring that each link has at most one pre-aggregated rollup row per calendar date.

Q102: How does AnalyticsQueryService retrieve dimension breakdowns?

Answer: `_aggregate_dimension()` performs SQL grouping and aggregation using `queryset.values(field).annotate(count=Count('id')).order_by('-count')[:limit]`.

Q103: What dimension breakdown categories are calculated?

Answer: Top Countries (**country_code**), Top Devices (**device_type**), Top Browsers (**browser**), and Top Referrers (**referrer**).

Q104: What is drf-spectacular?

Answer: An OpenAPI 3.0 schema generation library for Django REST Framework that inspects serializers, views, and parameters to generate interactive Swagger UI (`/api/docs/`) and ReDoc (`/api/redoc/`) documentation.

Q105: How are Swagger API schemas customized in DRF views?

Answer: Using the `@extend_schema` decorator to document request bodies, parameters, tags, and response models.

Follow-up Cross-Questioning (Q106 - Q108):**Q106: Why is API documentation important for backend engineering projects?**

Answer: It acts as an authoritative contract between backend developers, frontend teams, and third-party API consumers, detailing endpoints, payload types, error codes, and authentication methods.

Q107: What is the difference between Swagger UI and ReDoc?

Answer: Swagger UI provides an interactive interface allowing users to execute live API calls (**Try it out**). ReDoc provides a clean, three-panel static documentation layout focused on readability.

Q108: What is django-prometheus?

Answer: A Django library that exports framework-level metrics (DB query execution count, view execution duration, HTTP response status codes) via endpoint `/metrics`.

Q109: What is a Kafka Topic?

Answer: A named channel or log stream in Kafka where producers publish records and consumers pull records.

Q110: What is a Kafka Partition?

Answer: A Topic is split into multiple Partitions distributed across brokers. Partitions enable horizontal scale, parallel processing, and ordered log commits.

Q111: How does LinkStream partition click events in Kafka?

Answer: **ClickEventProducer** partitions messages using `key=str(link_id).encode('utf-8')`. All click events for the same short link land on the same partition, guaranteeing strict ordering per link.

Q112: What is a Kafka Consumer Group?

Answer: A group of consumer workers cooperating to read data from a topic. Kafka assigns distinct topic partitions to distinct consumers within the group.

Q113: What happens if a consumer worker in a group crashes?

Answer: Kafka triggers a **Rebalance**. It reassigns the partitions previously owned by the failed consumer to the remaining active consumers in the group.

Q114: What is Kafka message serialization?

Answer: Converting in-memory Python dictionaries into raw byte streams (e.g., JSON UTF-8 bytes) before transmitting across the network.

Q115: What parameter controls Kafka producer acknowledgment?

Answer: acks=1. The leader broker writes the record to its local log and responds to the producer without waiting for full replica acknowledgments, balancing latency and durability.

Follow-up Cross-Questioning (Q116 - Q118):**Q116: What is the difference between acks=0, acks=1, and acks=all (acks=-1) in Kafka?**

Answer:

- **acks=0:** Producer does not wait for acknowledgment (highest throughput, risk of data loss).
- **acks=1:** Producer waits for leader broker acknowledgment (medium throughput, resilient to follower failure).
- **acks=all:** Producer waits for leader and all in-sync replicas (ISR) to acknowledge (highest durability, higher latency).

Q117: What is the linger.ms setting in Kafka Producer?

Answer: linger.ms=10 instructs the producer to wait up to 10ms before sending a batch of messages, allowing small requests to combine into larger, network-efficient batches.

Q118: What is max.block.ms in Kafka Producer?

Answer: max.block.ms=500 limits the maximum time `producer.send()` will block if Kafka metadata is unavailable or internal buffers are full, after which it falls back gracefully.

Q119: What is PostgreSQL B-Tree index, and why is it used for short_code?

Answer: B-Tree (Balanced Tree) is a self-balancing tree data structure that maintains sorted data. It provides $O(\log N)$ time complexity for equality (=) and range queries (<, >), ideal for **short_code** string lookups.

Q120: What is a Composite Index in database systems?

Answer: An index created on multiple columns combined (e.g., `(user_id, -created_at)`). It allows PostgreSQL to satisfy multi-column **WHERE** and **ORDER BY** clauses in a single index scan without sorting in memory.

Q121: What is the Leftmost Prefix Rule in Composite Indexes?

Answer: A composite index on **(A, B)** can satisfy queries filtering by **(A)** or **(A, B)**, but cannot be used for queries filtering only by **(B)**.

Q122: What is database connection pooling, and why is CONN_MAX_AGE set in settings.py?

Answer: Connection pooling reuses open database TCP connections across multiple HTTP requests. **CONN_MAX_AGE = 60** keeps database connections open for 60 seconds, eliminating connection setup overhead on every request.

Q123: What is the difference between BIGINT and INTEGER in PostgreSQL?

Answer: INTEGER is a 4-byte integer (up to ~2.14 Billion). **BIGINT** is an 8-byte integer (up to ~9.22 Quintillion). **click_count** uses **BIGINT** to prevent integer overflow on high-traffic links.

Q124: What is the purpose of `db.sqlite3` in local development?

Answer: SQLite is a serverless, zero-configuration file-based database used for fast local development and lightweight automated testing without running a dedicated PostgreSQL server.

Q125: How does `settings.py` automatically detect database configuration?

Answer: `is_service_available(DB_HOST, DB_PORT)` tests TCP socket connection to PostgreSQL port 5432. If unavailable, it sets `USE_SQLITE = True` and switches to `db.sqlite3`.

Follow-up Cross-Questioning (Q126 - Q128):**Q126: Why is SQLite NOT suitable for high-concurrency production deployments?**

Answer: SQLite uses file-level write locking. Multiple concurrent write operations lock the entire database file, causing severe lock contention and **database is locked** errors under concurrent write traffic.

Q127: What is database sharding?

Answer: Horizontal partitioning of a database across multiple physical server nodes based on a shard key (e.g., `hash(short_code) % N_shards`).

Q128: How would you shard LinkStream if link storage grew to 10 Billion rows?

Answer: Shard the **links** table across multiple PostgreSQL database clusters using `hash(short_code)` as the shard key, using a database router or proxy like Vitess / Citus DB.

Q129: What is Cross-Site Scripting (XSS), and how does LinkStream protect against it?

Answer: XSS occurs when malicious JavaScript is injected into web pages. React automatically escapes HTML contents rendered in JSX, and Django sanitizes headers and template rendering.

Q130: What is Cross-Site Request Forgery (CSRF), and why are stateless JWT APIs immune to traditional CSRF?

Answer: CSRF tricks authenticated browsers into making unauthorized requests via auto-sent cookies. LinkStream uses JWT tokens stored in JS memory / headers (**Authorization: Bearer**), which browsers do not automatically attach to cross-site requests.

Q131: What is HTTP Strict Transport Security (HSTS)?

Answer: A security header (**Strict-Transport-Security**) that forces browsers to communicate with the server exclusively via encrypted HTTPS connections.

Q132: What is Content Security Policy (CSP)?

Answer: An HTTP header that restricts the sources from which scripts, styles, images, and frames can be loaded, mitigating XSS and data injection attacks.

Q133: What is the difference between authentication and authorization?

Answer: Authentication verifies *who the user is* (e.g., email & password login). Authorization determines *what resources the authenticated user is allowed to access* (e.g., **IsOwner** permission class).

Q134: How does Django handle password validation?

Answer: Via `AUTH_PASSWORD_VALIDATORS` in `settings.py`, validating minimum length (8+ chars), checking against common passwords, checking for all-numeric patterns, and verifying user attribute dissimilarity.

Q135: What is `django-cors-headers` middleware, and why must it be placed high in MIDDLEWARE order?

Answer: It appends CORS response headers (**Access-Control-Allow-Origin**). It must be placed before view handling so CORS headers are included even on early exception responses.

Follow-up Cross-Questioning (Q136 - Q138):

Q136: What is a Denial of Service (DoS) Slowloris attack?

Answer: An attack where the client opens multiple HTTP connections and sends headers very slowly, holding server sockets open until worker pools are exhausted. Nginx mitigates this using `client_body_timeout` and `client_header_timeout`.

Q137: How does Nginx protect Gunicorn from slow clients?

Answer: Nginx acts as a reverse proxy buffer. It reads the complete slow HTTP request from the client into memory before opening a fast local socket connection to Gunicorn.

Q138: What is rate limit graceful degradation?

Answer: If the rate limiting store (Redis) fails, the rate limiter fails open (or falls back to local memory), prioritizing system availability over strict quota enforcement.

Q139: What is the role of Gunicorn in Docker container deployment?

Answer: Gunicorn serves as the production WSGI application server inside the **backend** container, handling HTTP requests dispatched by Nginx.

Q140: What is the command used to start Gunicorn in backend/Dockerfile?

Answer: `gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 4 --threads 2`.

Q141: What is multi-stage Docker build, and why is it used for the frontend?

Answer: Multi-stage builds use separate **FROM** stages. Stage 1 (**node:20-alpine**) compiles React source code into static asset bundles via `npm run build`. Stage 2 (**nginx:alpine**) copies only the compiled `/dist` directory, producing a lightweight (~20MB) production image.

Q142: What is the contents of frontend/nginx.conf?

Answer: It configures Nginx to serve static files from `/usr/share/nginx/html`, routes SPA fallback requests via `try_files $uri $uri/ /index.html`; and proxies `/api/` calls to `http://backend:8000/api/`.

Q143: What is Docker Volume, and why are volumes defined for postgres_data and redis_data?

Answer: Containers are ephemeral (data is erased on container stop). Docker Volumes persist database files on the host filesystem independently of container lifecycles.

Q144: What is Docker Network, and how do containers communicate in docker-compose.yml?

Answer: Docker Compose creates an isolated virtual bridge network. Services resolve each other by container service name (e.g., `http://postgres:5432`, `redis://redis:6379`).

Q145: What is .env.example?

Answer: A template file committed to version control that documents required environment variables without exposing sensitive production secrets.

Follow-up Cross-Questioning (Q146 - Q148):**Q146: Why should .env files NEVER be committed to Git?**

Answer: `.env` files contain sensitive credentials (API secret keys, database passwords, JWT signing keys). Committing secrets to Git exposes them to security leaks.

Q147: How does .gitignore prevent accidental secret commits?

Answer: It lists file patterns (`.env`, `*.log`, `db.sqlite3`, `node_modules/`, `venv/`) that Git explicitly ignores during `git add`.

Q148: What is GitHub CLI (gh), and how was it used to publish this repository?

Answer: `gh` is GitHub's official command-line tool. It was used to execute `gh repo create scalable-url-shortener-analytics --public --source=. --push`, creating and pushing the repo directly from the shell.

Q149: What is Prometheus scraping?

Answer: Prometheus uses a pull model. It periodically issues HTTP **GET** `/metrics` requests to target services to retrieve metrics formatted in Prometheus text exposition format.

Q150: What is PromQL?

Answer: Prometheus Query Language, used to query time-series metrics, calculate rates over time (`rate()`), and aggregate metrics across dimensions.

Q151: Write a PromQL query to calculate the 5-minute average HTTP request rate per second.

Answer: `sum(rate(redirect_requests_total[5m]))`.

Q152: What is Grafana Provisioning?

Answer: Automated configuration of Grafana datasources (`datasource.yml`) and dashboards (`dashboard.yml`) from YAML/JSON files on container boot, eliminating manual UI configuration.

Q153: What is the purpose of `django_prometheus.urls` in `urls.py`?

Answer: Exposes `/metrics` endpoint for Prometheus scraping.

Q154: What is a Prometheus Counter vs Histogram?

Answer:

- **Counter:** A cumulative metric that only increases (e.g., `redirect_requests_total`).
- **Histogram:** Samples observations into configurable latency buckets (e.g., `redirect_latency_seconds`).

Q155: What is Grafana Alerting?

Answer: Rules configured in Grafana that evaluate PromQL queries and dispatch notifications (PagerDuty, Slack, Email) when thresholds are breached.

Follow-up Cross-Questioning (Q156 - Q158):**Q156: Why does Prometheus use a Pull model instead of a Push model?**

Answer: The pull model allows Prometheus to control scraping frequency, detect target down states automatically (if target fails to respond to HTTP scrape), and prevents targets from overwhelming the metrics server.

Q157: What is Pushgateway in Prometheus?

Answer: An intermediary metric cache used for short-lived batch jobs (like Celery scripts) that exit before Prometheus can scrape them.

Q158: What metric detects consumer lag in Kafka?

Answer: `kafka_consumergroup_lag` measured by Prometheus Kafka Exporter.

Q159: What is pytest?

Answer: A Python testing framework providing clean syntax, fixture management, and assertions.

Q160: What is pytest-django?

Answer: A plugin supplying Django-specific test fixtures (`db`, `client`, `settings`, `rf`).

Q161: What does `@pytest.mark.django_db` decorator do?

Answer: Instructs pytest to set up a clean, isolated test database for the duration of the test.

Q162: What is unit testing vs integration testing?

Answer:

- **Unit Testing:** Tests individual functions in isolation (e.g., `base62.encode()`).

• **Integration Testing:** Tests interactions across multiple modules (e.g., API request $\rightarrow$ Serializer $\rightarrow$ ORM $\rightarrow$ Response).

Q163: How are HTTP API calls tested in `test_auth.py` and `test_links.py`?

Answer: Using DRF's `APIClient()` fixture to dispatch HTTP requests (`client.post()`, `client.get()`) and assert response status codes and JSON payloads.

Q164: How are mocks used in `test_cache.py`?

Answer: `unittest.mock` / `pytest-mock` patches `cache.get` and `cache.set` methods to verify cache hits and misses without requiring a live Redis server.

Q165: What test file tests Base62 encoding?

Answer: `backend/tests/test_links.py`.

Follow-up Cross-Questioning (Q166 - Q168):

Q166: What is Code Coverage, and how is it measured in Python?

Answer: Code coverage measures the percentage of source code executed during test suite runs. It is measured using `coverage.py` (`pytest --cov=apps`).

Q167: What is Test-Driven Development (TDD)?

Answer: A development workflow: Write a failing test first (Red), implement code to make the test pass (Green), and then refactor code (Refactor).

Q168: How do you mock Kafka publishing during pytest execution?

Answer: `settings.IS_TESTING = True` instructs `ClickEventProducer.get_producer()` to return `None`, bypassing Kafka network socket calls during test execution.

Q169: What is Base62 character set order?

Answer: `0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ` (62 characters).

Q170: Convert integer 0 to Base62.

Answer: `'0'`.

Q171: What is the main advantage of Base62 over Base64 for URLs?

Answer: Base64 includes special characters `+` and `/` (and `=` padding), which have special meanings in URL paths and query parameters. Base62 contains only URL-safe alphanumeric characters.

Q172: What is the maximum string length of a destination URL supported in `LinkCreateSerializer`?

Answer: 2048 characters (matching standard web browser maximum URL limits).

Q173: What is the default expiration time for Redis link cache keys?

Answer: 86,400 seconds (24 hours).

Q174: What is the purpose of `manage.py` in Django?

Answer: A command-line script for executing Django management commands (`runserver`, `migrate`, `makemigrations`, `createsuperuser`).

Q175: What command creates new database migrations after modifying `models.py`?

Answer: `python manage.py makemigrations`.

Follow-up Cross-Questioning (Q176 - Q178):

Q176: What command applies pending migrations to the database?

Answer: `python manage.py migrate`.

Q177: What is Django migration dependency tracking?

Answer: Django records applied migration files in the `django_migrations` table to ensure migrations execute in strict order across dependencies.

Q178: What happens if two developers create conflicting migrations on the same branch?

Answer: Django raises a `CommandError: Conflicting migrations detected`. Resolved by running `python manage.py makemigrations --merge`.

Q179: What is React Virtual DOM?

Answer: A lightweight in-memory representation of the real DOM. React compares Virtual DOM trees (Reconciliation) and updates only changed elements in the real DOM.

Q180: What is useState hook in React?

Answer: A hook allowing functional components to manage local state variables.

Q181: What is useEffect hook in React?

Answer: A hook for executing side effects (data fetching, subscriptions, timers) in functional components.

Q182: What is useCallback hook in React?

Answer: A hook that memoizes function definitions across re-renders to prevent unnecessary child component re-renders.

Q183: What is useParams hook in React Router?

Answer: A hook that extracts dynamic URL path parameters (e.g., `LinkId` from `/analytics/:LinkId`).

Q184: What is useNavigate hook in React Router?

Answer: A hook providing programmatic navigation between routes (e.g., `navigate('/login')`).

Q185: What is ProtectedPage component in App.jsx?

Answer: A higher-order wrapper component that checks if user is authenticated; if `user == null`, it renders `<Navigate to="/login" replace />`.

Follow-up Cross-Questioning (Q186 - Q188):

Q186: What is Prop Drilling in React, and how can it be avoided?

Answer: Prop drilling passes props through multiple levels of nested components. Avoided using React Context API or Redux state management.

Q187: What is React.StrictMode?

Answer: A development tool that highlights potential problems in an application by running render functions twice to catch side effects.

Q188: What is Hot Module Replacement (HMR)?

Answer: A Vite feature that updates modified modules in the browser instantly without refreshing state or reloading the entire page.

Q189: What is the current capacity of LinkStream's 7-character Base62 keyspace?

Answer: 3.52 Trillion links.

Q190: What is the current bottleneck if traffic scales from 1K QPS to 100K QPS?

Answer: Single-node PostgreSQL write contention and Redis network bandwidth.

Q191: How would CDN Edge Redirection optimize the platform at 100K QPS?

Answer: Cache short URL mappings at CDN Edge Workers (Cloudflare Workers / AWS CloudFront KeyValueCollection), returning redirects at the network edge in < 5ms without hitting origin servers.

Q192: What is TimescaleDB, and why is it recommended for scaling click events?

Answer: An open-source time-series extension for PostgreSQL. It automatically partitions tables into time-based chunks (Hypertables), speeding up analytical queries.

Q193: What is Log Compaction in Apache Kafka?

Answer: A retention policy where Kafka retains the latest record for each message key within a partition, acting as a state store.

Q194: What is an In-Memory LRU Cache?

Answer: Least Recently Used cache evicts the items accessed longest ago when capacity limits are reached.

Q195: What is a Circuit Breaker pattern in distributed systems?

Answer: A pattern that detects downstream service failures (e.g., Redis down) and immediately rejects or reroutes requests without waiting for timeout periods.

Follow-up Cross-Questioning (Q196 - Q200):

Q196: What is a Thundering Herd Problem on database startup?

Answer: When a database restarts, cold caches cause all incoming requests to hit the DB simultaneously. Mitigated by cache warming scripts.

Q197: What is Database Connection Starvation?

Answer: When all available database pool connections are exhausted by slow queries, blocking new API requests.

Q198: What is a Backpressure Mechanism in reactive streams?

Answer: A control flow mechanism where consumers signal producers to slow down data transmission when consumer queues become full.

Q199: Summarize the complete technical architecture of LinkStream in one sentence.

Answer: LinkStream is a high-performance, event-driven URL shortener that achieves sub-10ms redirects using Redis Cache-Aside caching while offloading click telemetry asynchronously to Apache Kafka and PostgreSQL batch workers under full Prometheus/Grafana observability.

Q200: If an examiner asks why you built this project, what is your closing response?

Answer: "I built LinkStream to master real-world distributed systems engineering—learning how to separate high-velocity read paths from analytical write streams, implement fault-tolerant caching and rate limiting, containerize complex microservices, and instrument full-stack observability for high-concurrency production systems."